



Strong Data Integrity Checks in Erasure Coding(EC) Storage Systems

Uma Maheswara Rao Gangumalla

Sr. Engineering Manager - Storage, Cloudera Inc

Aswin Shakil Balasubramanian

Software Engineer II, Cloudera Inc

Swaminathan Balachandran

Senior Software Engineer- Storage, Cloudera Inc

Speakers Intro

Uma Maheswara Rao G

- ❑ Sr. Engineering Manager at Cloudera
- ❑ ASF Member. A Committer and PMC member on Apache Hadoop, Ozone, Incubator, Ratis...etc
- ❑ Mentor @ Incubator
- ❑ ApacheCon/CoC Track Chair - 2021, 2022, 2023

Aswin Shakil Balasubramanian

- ❑ Software Engineer II at Cloudera
- ❑ Apache Ozone Contributor
- ❑ Apache Ozone Project Committer

Swaminathan Balachandran

- ❑ Senior Software Engineer at Cloudera
- ❑ Apache Ozone Contributor
- ❑ Apache Ozone Project Committer

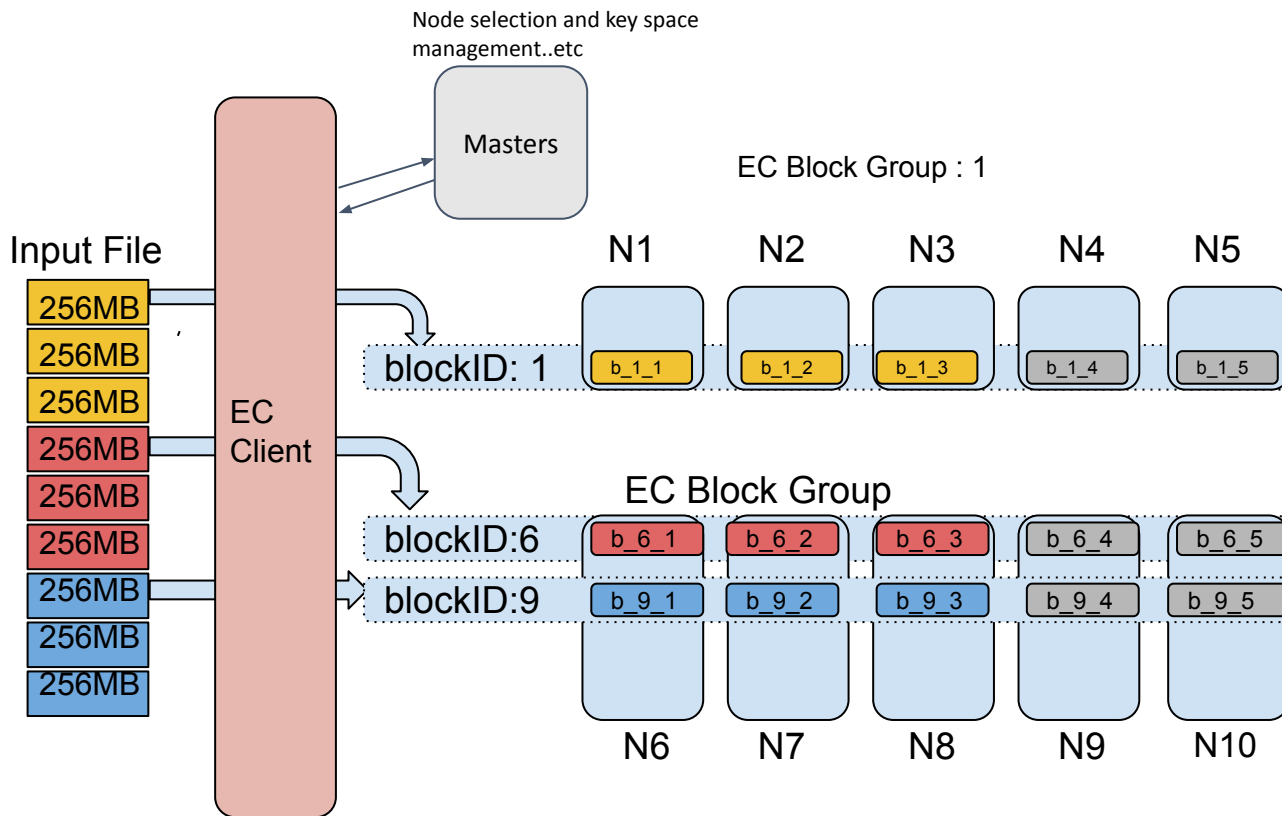
Agenda

- ❑ Quick Overview - How EC Works
 - ❑ Write
 - ❑ Read
 - ❑ Offline Recovery
- ❑ Reconstruction time corruption possibilities
 - ❑ Detection techniques & challenges
- ❑ Proposed solution
- ❑ Question and Answers

Terminology and Implementation Considerations

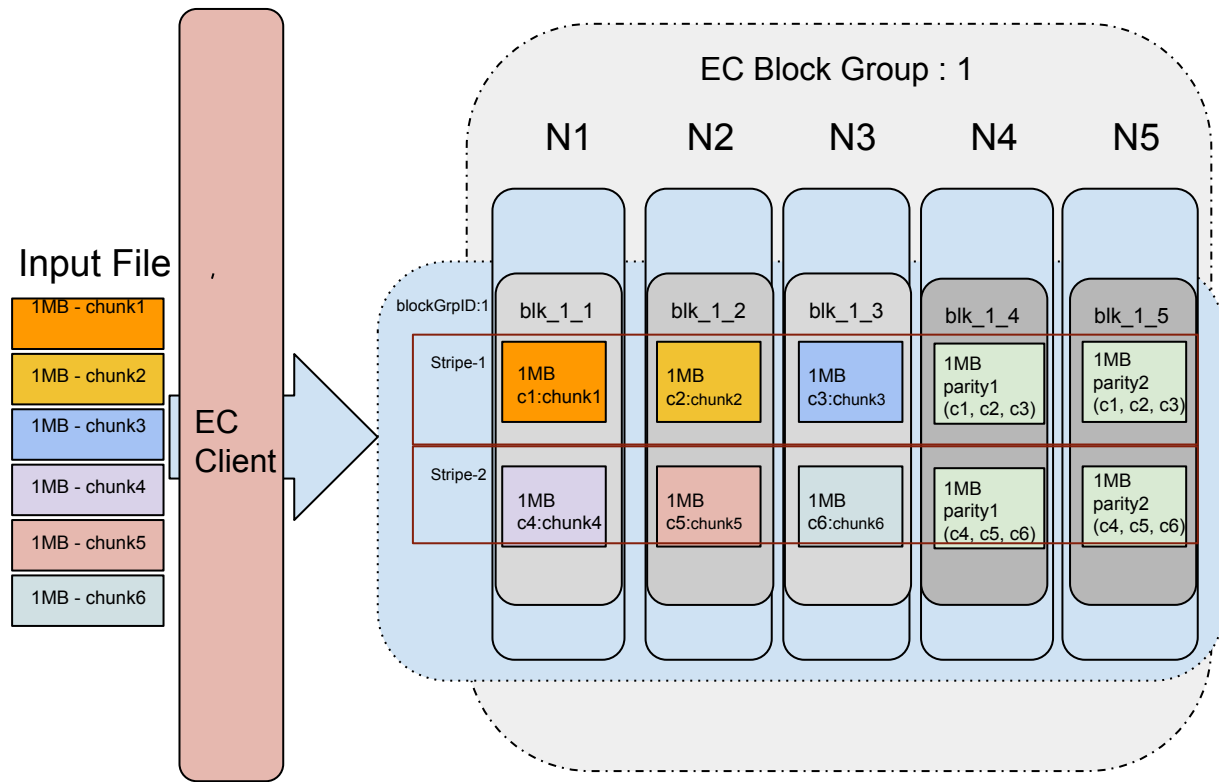
- Block Group: a block presents in a container group.
- Each data(d) + parity(p) number of chunks written to block group to finish 'Stripe'.
- Parity generated at the client.

Write



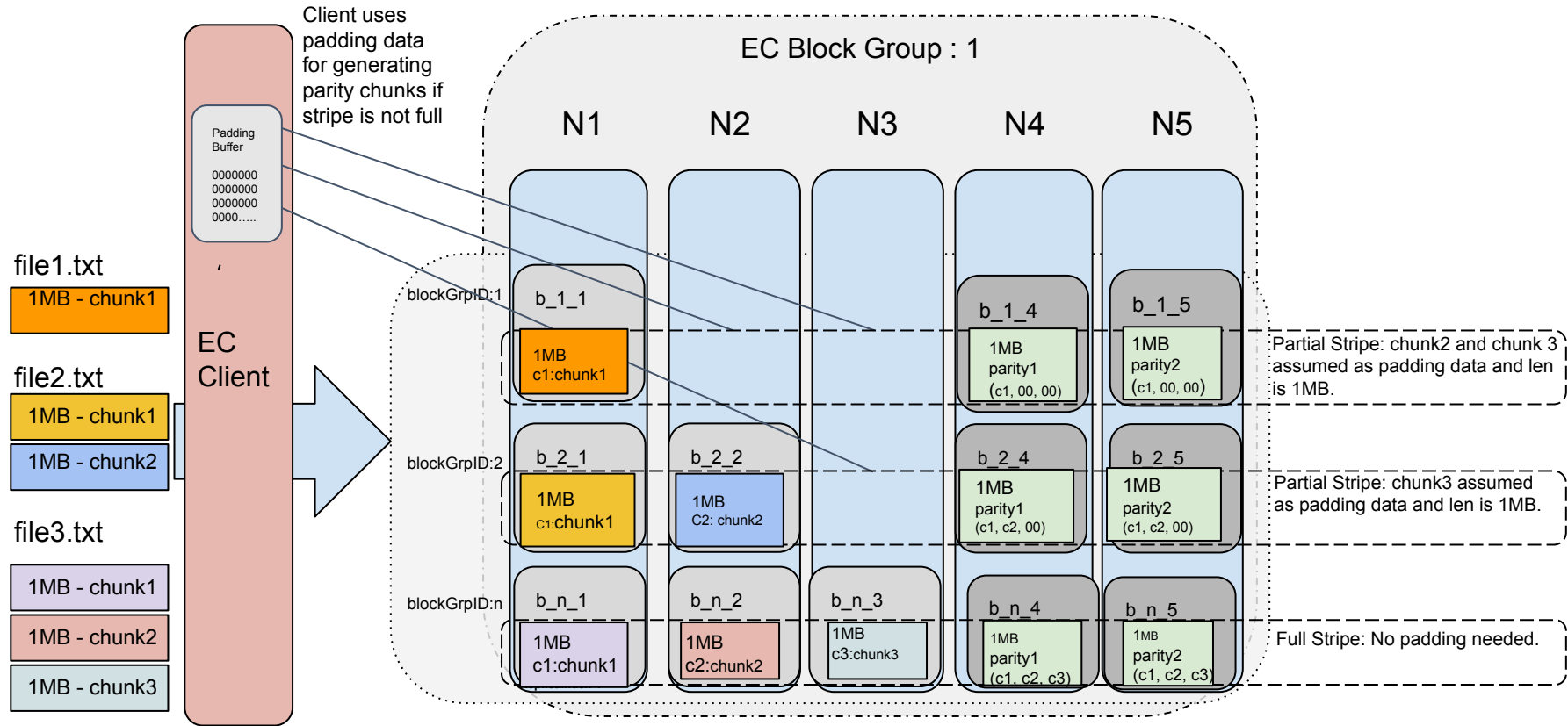
- Placement policies pick the required number of nodes for EC.
- Nodes set will be chosen based on EC schema (3(d):2(p), 6:3, 10:4).
- After finish writing d* block_size data, client requests masters to get new node set.

Write: Striping



- Stripe: One round of data + parity chunks called as full stripe.
- Chunks would be written in round robin fashion to data nodes.
- Parity Generation: After every data number of chunks written, parity will be generated and send to remaining nodes in group.
- If stripe write fails, the current block group will be closed and rewrite the failed stripe to new block group.
- Client keep track of bytes written and check for failures.

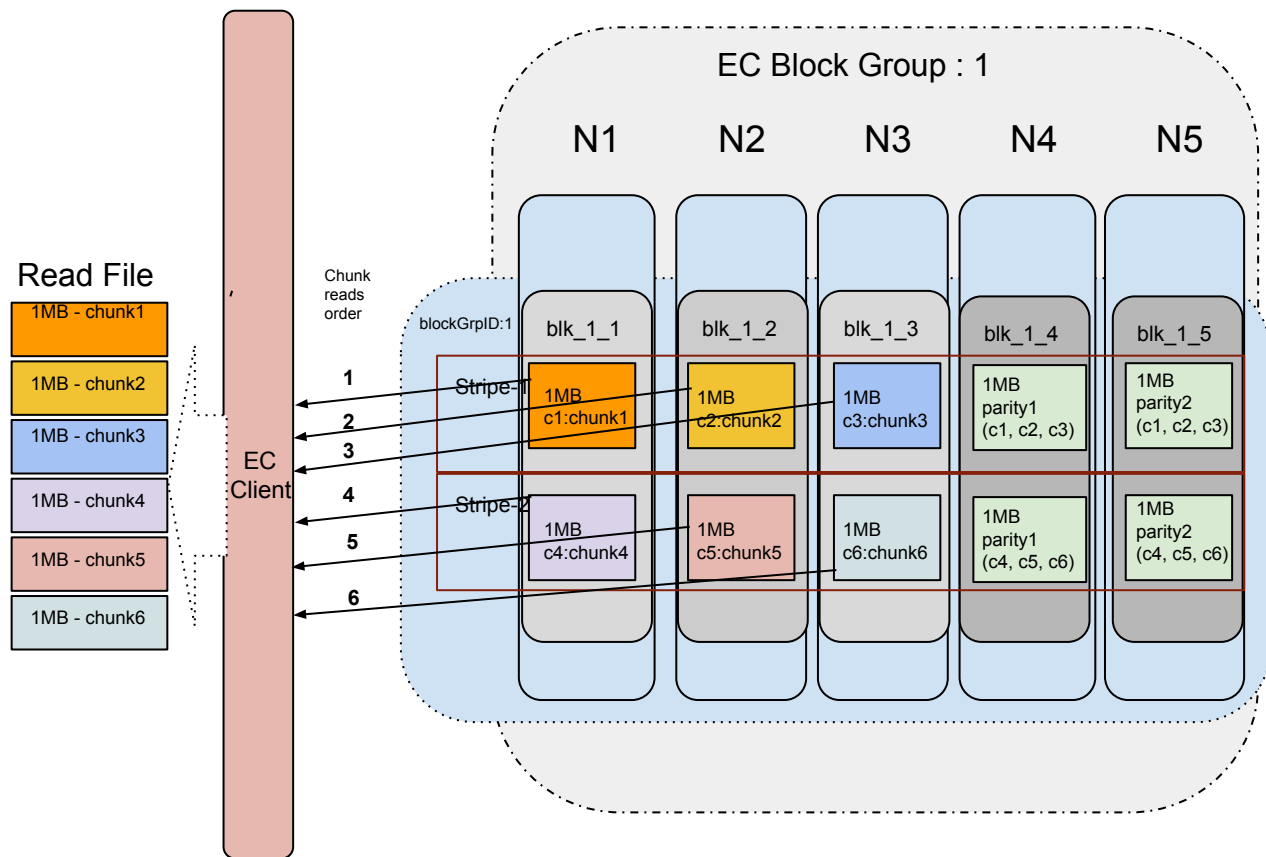
Write: Partial Stripe with Padding



Write: Striping

- If stripe write fails, the current block group will be closed
 - Failed stripe will be rewritten to the new block group.
- Client keep track of bytes written and check for failures.
- After all data writes finishes, then parity writes.

Read

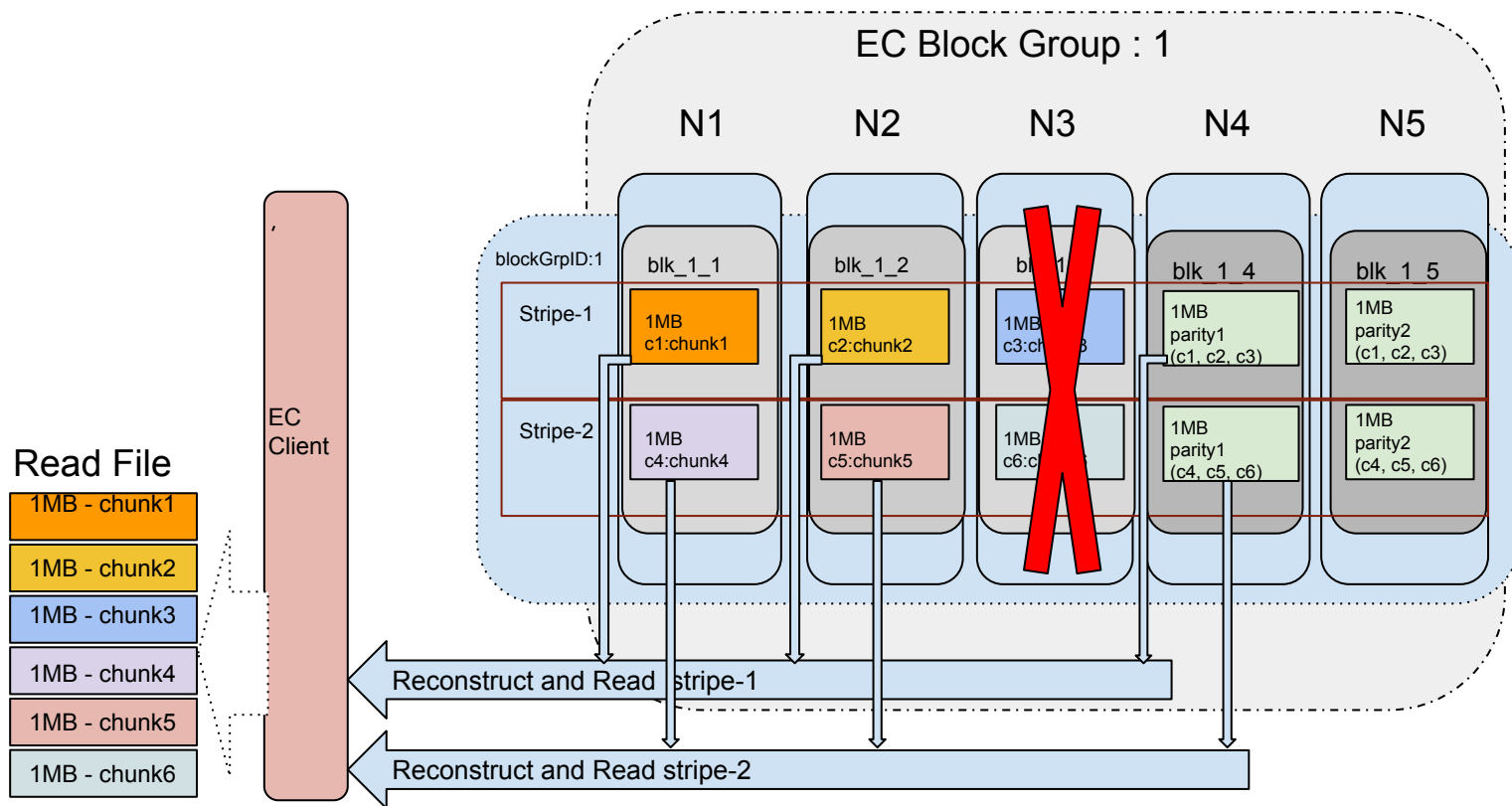


- Reads in the same order in which order writes done. Order will be based on replica Indexes.
- Client stitches the data back to original order and serves to user.

Reconstructional Reads

- First read will attempt to read data blocks.
- When node failed while reading, client will switch to reconstructional read and read from parity and reconstruct the lost data transparently.
- Reconstruction read will have overhead due to ec decode operation.
- To avoid the degraded reads, we need to recover the lost replicas offline.

Recovery Reads

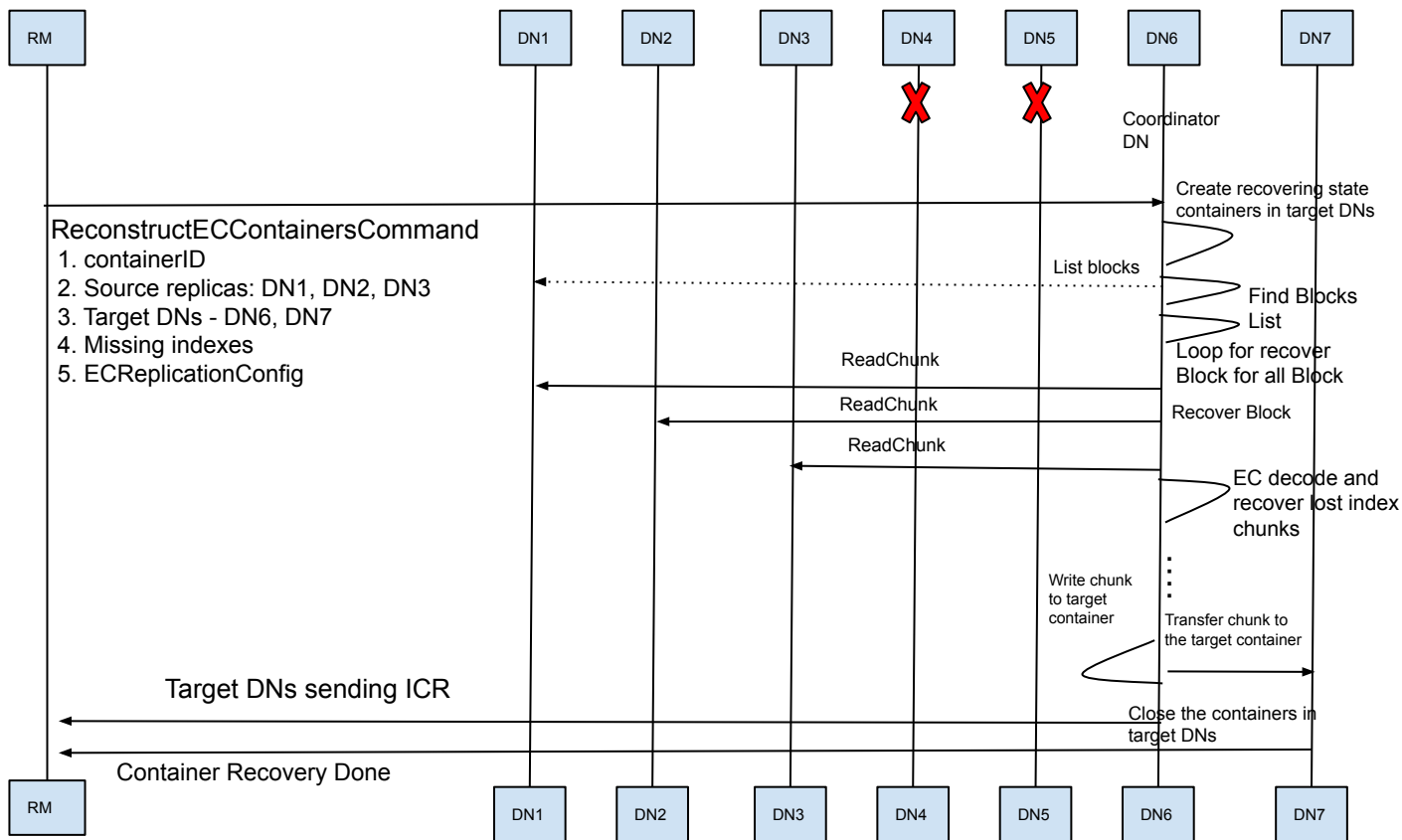


Offline Recovery

What is the Offline Recovery?

- When a node/Disk lost, we will lose the containers which are residing in that node/disk.
- We need a mechanism to recover that lost containers in the background.
- We call this process of background recovery as “Offline Recovery”.
- This is very critical background task similar re-replication on node/disk failures.

Offline Recovery





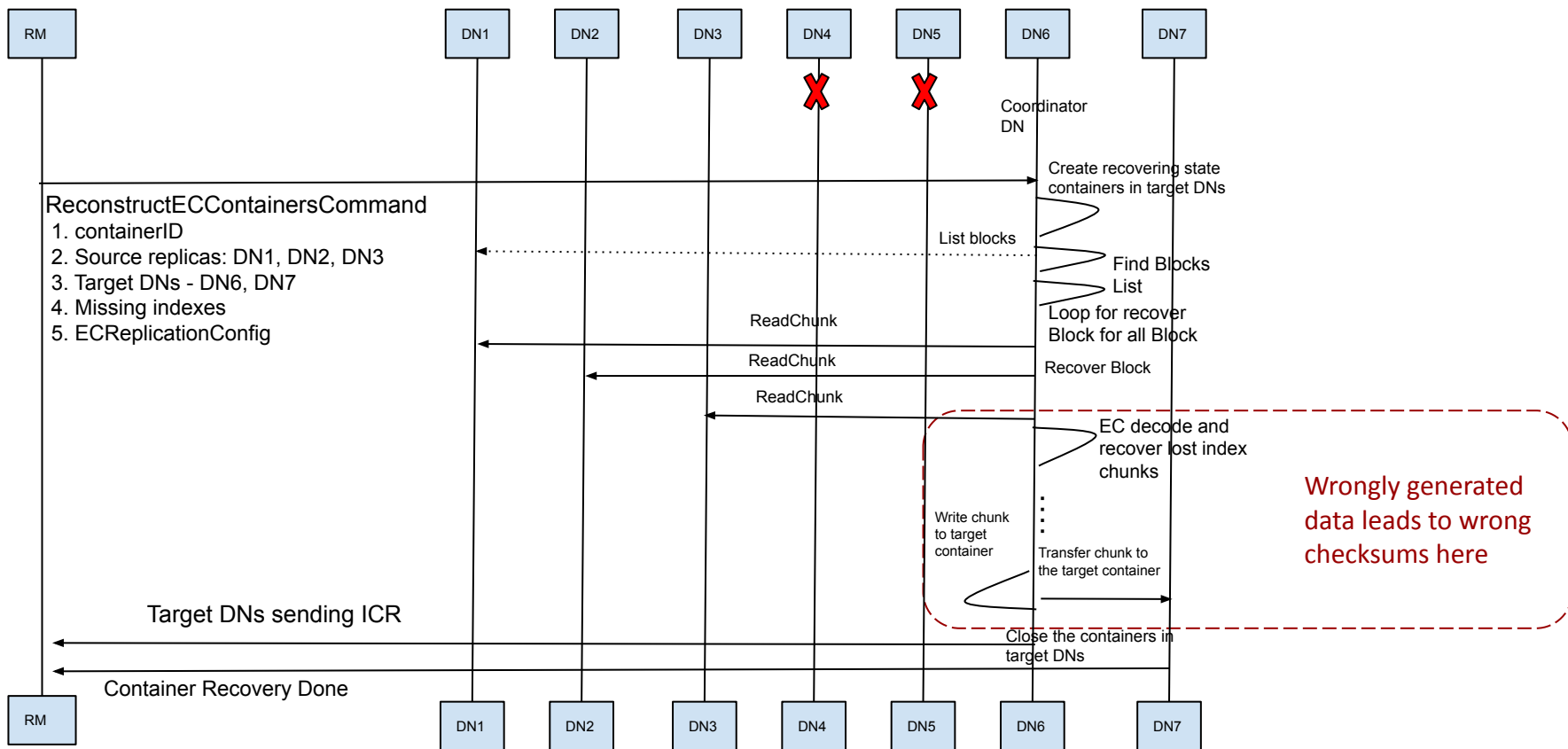
Reconstruction Time Corruption Possibilities



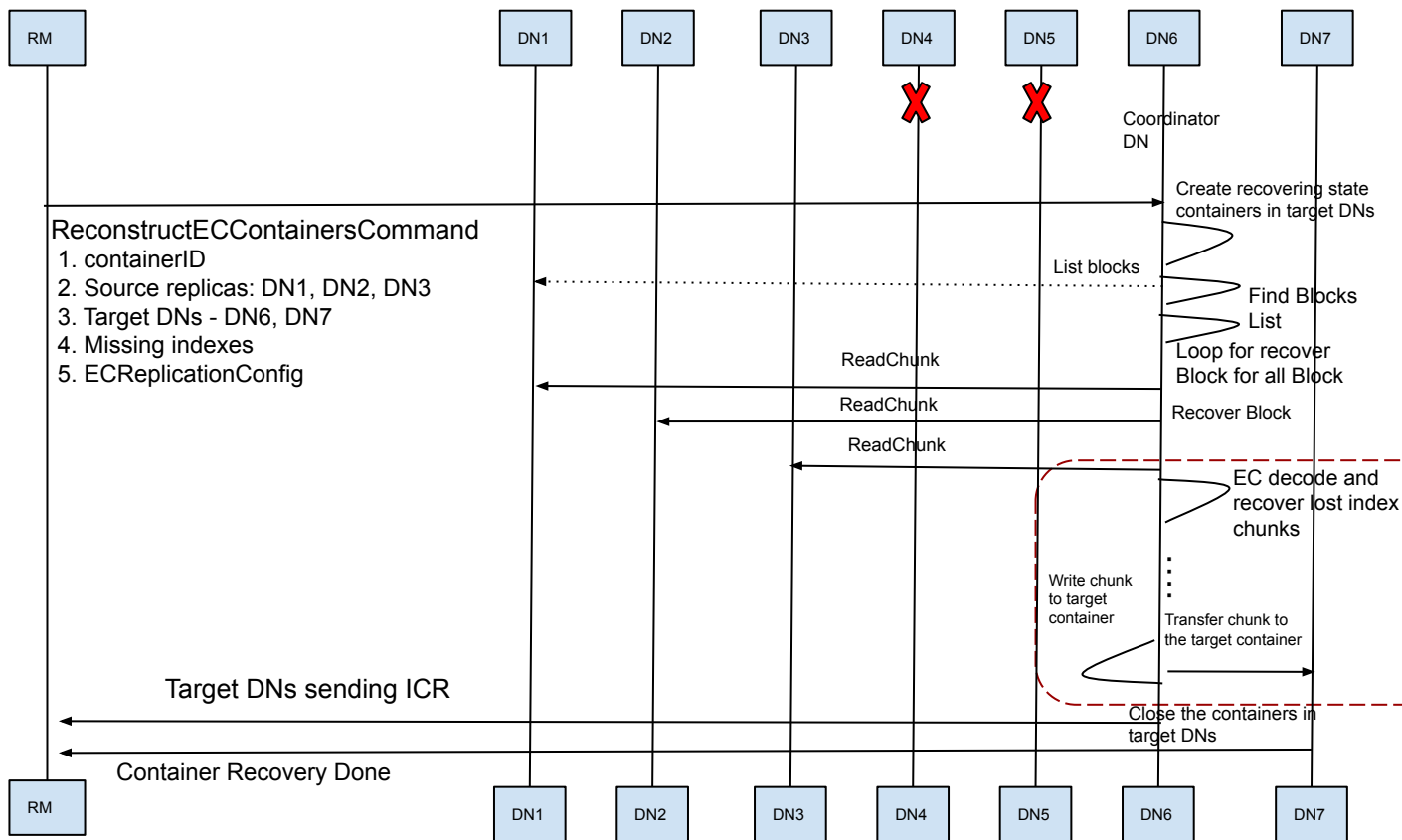
CoC NA
Oct. 7 – Oct. 10

2023

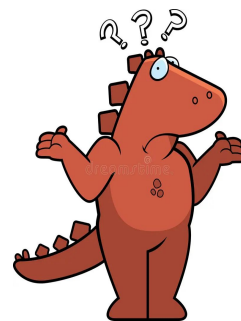
Offline Recovery



Offline Recovery



How do we detect?



Corruption Detection & Challenges

Here is one example corruption situation we dealt with.

- Some of blocks content turned out with all zeros.
 - Found the issue when compared with source data checksums.
- Need to find the blocks with all zeros as evidence of corruption :
<https://github.com/umamaheswararao/ec-corruption-analyzer>
- Need additional confirmation to make sure corruption is real :
<https://github.com/sodonnell/hdfs-ec-validator>
- Fixing of such corruption is hard, need to delete the blocks which are impacted in a block group.
- System will fix with right content automatically.
- If impacted blocks are more than d number of blocks, it is not possible to fix automatically unless we have source data available.



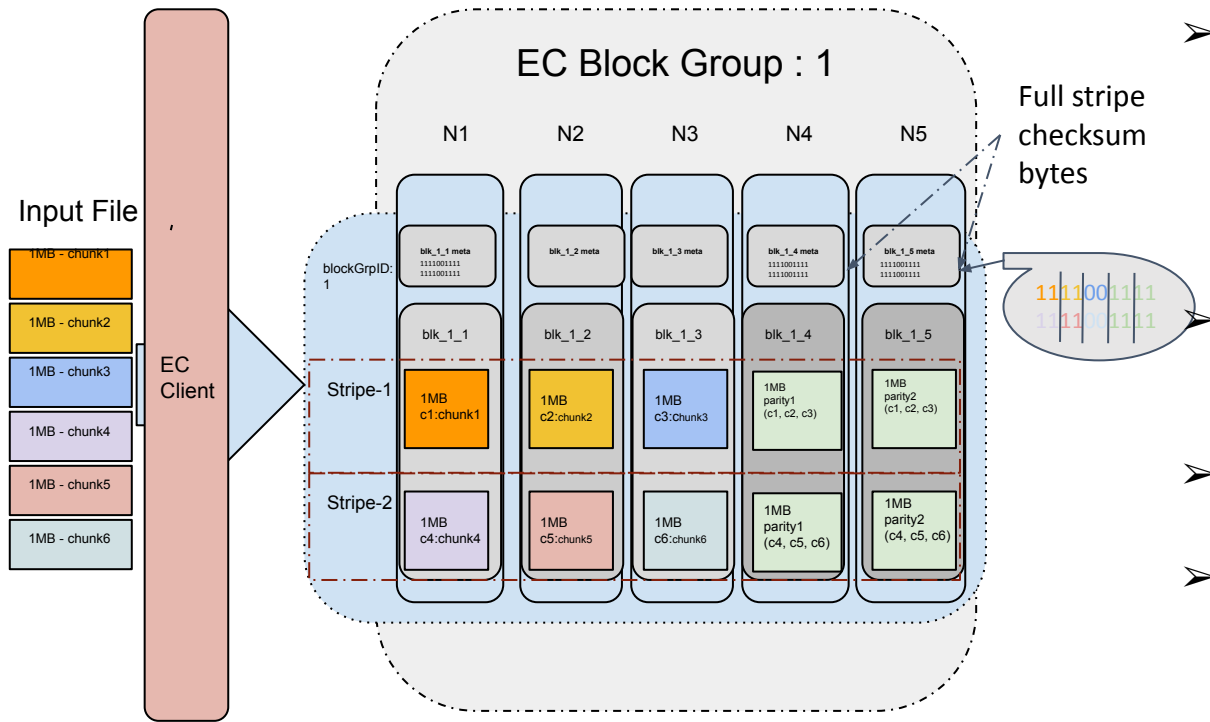
Proposed Solution



CoC NA
Oct. 7 – Oct. 10

2023

Solution (WIP)



- Checksum bytes calculated for full stripe at client.
- Checksum bytes are splittable at chunk boundaries. Meaning when checksum bytes are 4 bytes for chunk, full stripe checksum bytes are $5 * 4\text{bytes}$ (Note: in the interest of space picture only showing 2 bits checksum length for chunk)
- So, we can find specific chunk position checksum bytes from full stripe checksum bytes.
- We can store these checksum bytes only at $P + 1$ nodes.
- Why $P+1$ nodes to store checksums?
 - At any point of time, reconstruction needs d nodes.
 - So, at least one node of $P+1$ will participate in reconstruction.

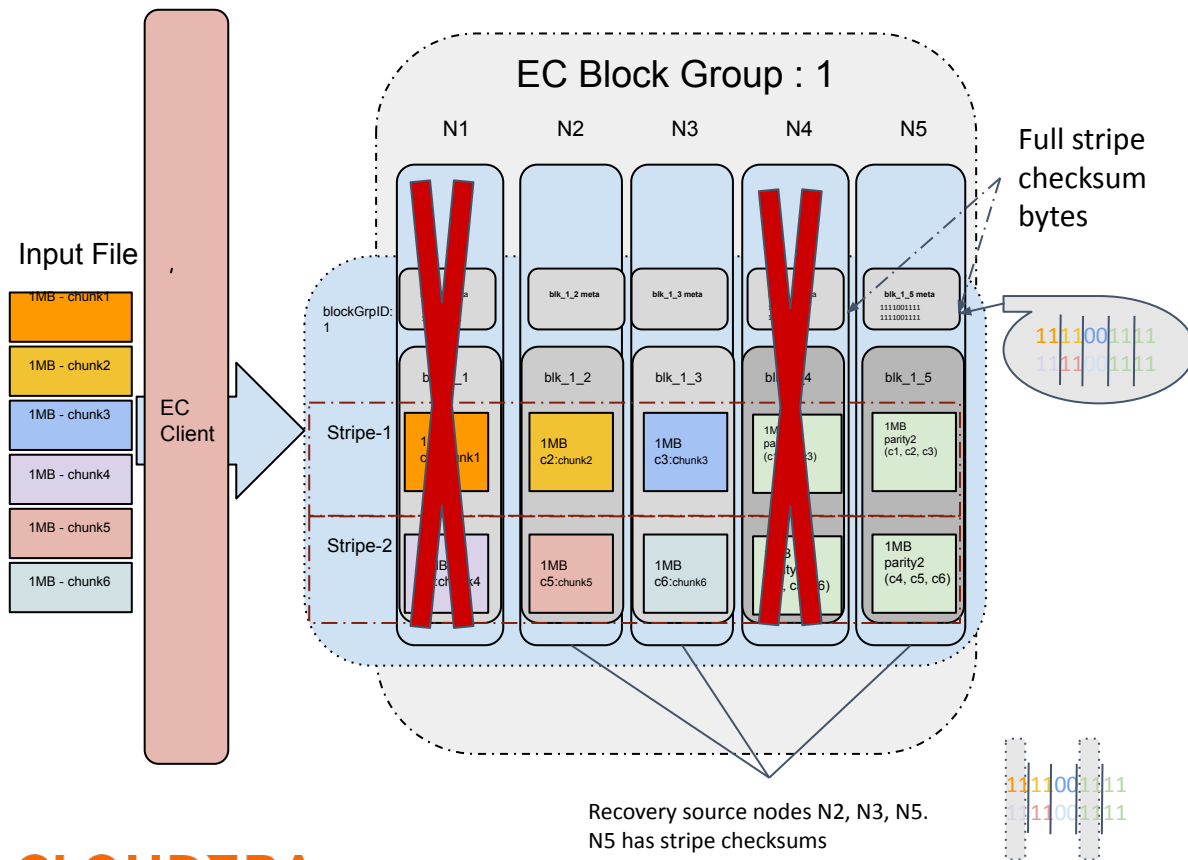
Pros

- getChecksum API can make sure of these precalculated stripe checksum
- When node lost, recovery node can use these checksum bytes to validate

Cons

- Little(negligible) overhead of storing these addition stripe checksum bytes at DN's

Recovery Time Validation



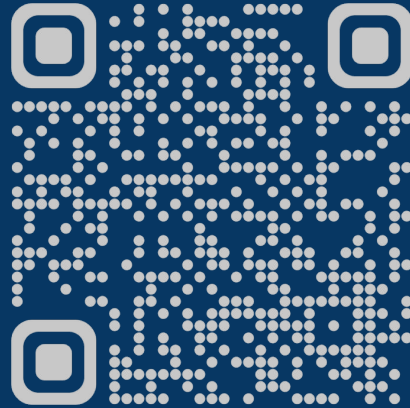
- N2, N3, N5 are used for reconstruction.
- On recovery of chunks, recovery node can use N5's original stripe checksum to validate with newly generated chunk checksum.
- If matches, chunk reconstruction is successful.
- If does not matches, reconstruction is wrong and recovery would fail.

EC supported in a scalable Object Storage: Apache Ozone

- ❑ Github repo: <https://github.com/apache/ozone>
- ❑ Looking to contribute to the Apache Ozone EC project?
 - ❑ Start with <https://github.com/apache/ozone/blob/master/CONTRIBUTING.md>
- ❑ Bug reporting is at: <https://issues.apache.org/jira/projects/HDDS>

Want to join a Storage Meetup?

- There is a storage meetup happening in Santa Clara on Oct 25.
- Scan the below QR code for details



Thanks

Q&A

umamahesh@apache.org

aswin@apache.com

sbalachandran@cloudera.com

Twitter: [@UmamaheswaraG](https://twitter.com/UmamaheswaraG)